Object Oriented Programming (CSEG1044)
Unit 06 – Lecture 04 Notes
Database Design + JDBC Layer

Tofik Ali

February 27, 2026

Contents

1 Lecture Overview **1**

2 Syllabus Alignment (Unit 06) **2**

3 Learning Outcomes **2**

4 Key Terms (Quick) **2**

5 Detailed Notes **2**
5.1 ER/entity model → relational schema 2
5.2 Keys and constraints (minimum) 2
5.3 JDBC layer (DAO pattern) 3

6 Worked Example: schema + DAO skeleton **3**

7 Deliverable-3 checklist **4**

8 In-Class Activity (evidence) **5**

9 Checkpoint Questions (with sample answers) **5**

1 Lecture Overview

Your Swing UI is not complete until data is saved permanently. This lecture converts your entity model into a database schema and creates the JDBC/DAO layer. Deliverable-3 is a big milestone: once DAO CRUD works, UI integration becomes easy.

Sources: Git SCM: *Pro Git* (2e) (version control workflow), accessed February 27, 2026.; GitHub Docs (repositories, issues, pull requests), accessed February 27, 2026.; Oracle Java Tutorials: Swing + JDBC (accessed February 27, 2026).; JUnit 5 User Guide (accessed February 27, 2026).; Apache Maven Project: Getting Started (accessed February 27, 2026).; Gradle User Manual: Getting Started (accessed February 27, 2026).; UML class diagrams: UML 2.x overview/spec (accessed February 27, 2026).

2 Syllabus Alignment (Unit 06)

Unit 06: database design + JDBC persistence

3 Learning Outcomes

After this lecture, you should be able to:

- Convert an ER/entity model into tables with keys and constraints.
- Implement JDBC DAO methods using `PreparedStatement` and `ResultSet` mapping.
- Complete Deliverable-3: DB schema + CRUD for 1–2 core entities.

4 Key Terms (Quick)

- **Primary key (PK)**: unique identifier for a row.
- **Foreign key (FK)**: reference to PK in another table.
- **Constraint**: rule enforced by DB (NOT NULL, UNIQUE, FK).
- **Index**: speeds up search (usually on PK/FK columns).
- **DAO**: class that performs CRUD using JDBC.
- **Mapping**: converting a DB row (`ResultSet`) into a Java object.

5 Detailed Notes

5.1 ER/entity model → relational schema

Start with the entities from Deliverable-2. For each entity:

- create a table,
- choose a primary key,
- choose column types (`INT`, `VARCHAR`, `TIMESTAMP`),
- add constraints (NOT NULL, UNIQUE),
- add foreign keys for relationships.

Example mapping. Booking has a resource → `bookings.resource_id` is a foreign key referencing `resources.id`.

5.2 Keys and constraints (minimum)

- Use integer auto-increment PKs or meaningful unique keys (like roll number) depending on domain.

- Use NOT NULL for mandatory fields.
- Use UNIQUE where duplicates are not allowed (e.g., resource name).
- Use FK constraints to preserve relationships.

5.3 JDBC layer (DAO pattern)

Recommended structure:

- `DbUtil`: provides DB connection.
- `ResourceDao`: CRUD for resources.
- `BookingDao`: CRUD for bookings.

Use `PreparedStatement` for all user-provided values. `try-with-resources` ensures connections and statements are closed properly.

Sources: Git SCM: *Pro Git* (2e) (version control workflow), accessed February 27, 2026.; GitHub Docs (repositories, issues, pull requests), accessed February 27, 2026.; Oracle Java Tutorials: Swing + JDBC (accessed February 27, 2026).; JUnit 5 User Guide (accessed February 27, 2026).; Apache Maven Project: Getting Started (accessed February 27, 2026).; Gradle User Manual: Getting Started (accessed February 27, 2026).; UML class diagrams: UML 2.x overview/spec (accessed February 27, 2026).

6 Worked Example: schema + DAO skeleton

Schema (example)

```
-- resources table
CREATE TABLE resources (
  id INT AUTO_INCREMENT PRIMARY KEY,
  name VARCHAR(100) NOT NULL UNIQUE,
  type VARCHAR(50) NOT NULL,
  capacity INT NOT NULL
);

-- bookings table
CREATE TABLE bookings (
  id INT AUTO_INCREMENT PRIMARY KEY,
  resource_id INT NOT NULL,
  user_name VARCHAR(100) NOT NULL,
  start_time TIMESTAMP NOT NULL,
  end_time TIMESTAMP NOT NULL,
  status VARCHAR(20) NOT NULL,
  CONSTRAINT fk_booking_resource FOREIGN KEY (resource_id)
    REFERENCES resources(id)
);
```

DbUtil (connection helper)

```
import java.sql.Connection;
import java.sql.DriverManager;
```

```java
import java.sql.SQLException;

public class DbUtil {
    private static final String URL = "jdbc:mysql://localhost:3306/crm";
    private static final String USER = "root";
    private static final String PASS = "password";

    public static Connection getConnection() throws SQLException {
        return DriverManager.getConnection(URL, USER, PASS);
    }
}
```

ResourceDao (CRUD skeleton)

```java
import java.sql.*;
import java.util.*;

public class ResourceDao {
    public void insert(Resource r) throws SQLException {
        String sql = "INSERT INTO resources(name, type, capacity) VALUES(?,?,?)";
        try (Connection con = DbUtil.getConnection();
             PreparedStatement ps = con.prepareStatement(sql)) {
            ps.setString(1, r.getName());
            ps.setString(2, r.getType());
            ps.setInt(3, r.getCapacity());
            ps.executeUpdate();
        }
    }

    public List<Resource> findAll() throws SQLException {
        String sql = "SELECT id, name, type, capacity FROM resources";
        List<Resource> out = new ArrayList<>();
        try (Connection con = DbUtil.getConnection();
             PreparedStatement ps = con.prepareStatement(sql);
             ResultSet rs = ps.executeQuery()) {
            while (rs.next()) {
                out.add(new Resource(
                    rs.getInt("id"),
                    rs.getString("name"),
                    rs.getString("type"),
                    rs.getInt("capacity")
                ));
            }
        }
        return out;
    }
}
```

7 Deliverable-3 checklist

- ER/schema diagram or table list (resources, bookings, users)

- SQL schema script committed in repo (`schema.sql`)
- DAO CRUD for 1–2 core entities (at least: insert + list)
- Simple test runner/main method that demonstrates DAO works

8 In-Class Activity (evidence)

- Commit `schema.sql` and DAO code.
- Create PR for DB module.
- Add issues for remaining DAO methods (update/delete/search).

9 Checkpoint Questions (with sample answers)

Q1. Why use foreign keys?

Sample answer: Foreign keys enforce relationships between tables, preventing invalid references (e.g., a booking referencing a resource that does not exist).

Q2. Why use `PreparedStatement`?

Sample answer: It supports parameterized queries, improves safety for user input, and avoids SQL injection risks compared to concatenating strings.